Here's every algorithm actually running in the code, mapped to what it's doing:

Stage	Algorithm	What it does	Where
Frame extraction	Fixed-rate sampling	Reads every Nth frame based on video FPS ÷ target FPS (2 fps default) — not a named algorithm, just a sampling heuristic to keep frame count manageable	frame_extraction.py
Frame quality	**Variance of Laplacian**	Standard blur-detection method — applies a Laplacian edge-detection kernel (cv2.Laplacian), then takes the variance of the result. Sharp images have high-variance edges; blurry ones don't	quality.py
Frame quality	Mean/std of pixel intensity	Not a named CV algorithm — plain statistics on the grayscale histogram (mean = exposure, std dev = contrast)	quality.py
Frame selection	**Farneback dense optical flow**	cv2.calcOpticalFlowFarneback — computes a per-pixel motion vector field between two frames; I take the median magnitude as a proxy for how much the camera has actually moved (parallax), to decide if a frame adds new information	selection.py
Feature extraction	**SIFT** (Scale-Invariant Feature Transform, Lowe 1999/2004)	Finds distinctive keypoints (corners, blobs) in each frame and describes each with a 128-dim descriptor that's stable across scale, rotation, and moderate lighting change	features.py → extract_sift()
Feature matching	**Brute-force matching + Lowe's ratio test**	cv2.BFMatcher compares every descriptor in frame A against every descriptor in frame B (L2 distance), keeps the top-2 nearest neighbors per point (knnMatch, k=2), then discards a match unless the best neighbor is meaningfully closer than the second-best (ratio < 0.75) — this is what filters out ambiguous/repetitive-texture matches	features.py → match_features()
Geometric verification	**RANSAC** (Random Sample Consensus) fitting an **essential matrix**	cv2.findEssentialMat(..., method=cv2.RANSAC) — repeatedly samples minimal point sets, fits a candidate essential matrix (5-point algorithm under the hood), and keeps the fit with the most inliers. This is what throws out bad matches — mismatches, moving objects, noise	features.py → estimate_relative_pose()

Stage	Algorithm	What it does	Where
Pose estimation	**Pose recovery via cheirality check**	cv2.recoverPose() decomposes the essential matrix into the 4 possible (R, t) solutions and picks the one where triangulated points actually land in front of both cameras	features.py → estimate_relative_pose()
Error metric	Symmetric epipolar transfer error	Not a separate algorithm — a diagnostic I compute from the fundamental matrix ($F = K^{-T}EK^{-1}$) to report how well the recovered geometry fits the inlier points, in pixels	features.py → estimate_relative_pose()
Camera intrinsics	FOV-based focal length approximation	A formula, not an algorithm — estimates focal length from image diagonal and an assumed 84° FOV when no real calibration is supplied	features.py → build_intrinsics()

So the two "named, textbook" algorithms doing the heavy lifting are **SIFT** (what to look for) and **RANSAC** (how to throw out the matches that are wrong) — everything else is either a well-known supporting technique (Laplacian variance, Farneback flow, Lowe's ratio test) or a small formula/heuristic I wrote to glue them together.

p